

Data Engineering Fundamentals — Quick Notes

1. Python

Core types: `int`, `float`, `str`, `bool`, `list`, `tuple`, `set`, `dict`

python

```
row = {"id": 1, "amount": 49.99, "status": "shipped"} # dict = one record
rows = [row, {"id": 2, "amount": 19.99, "status": "pending"}] # list of dicts

for r in rows:
    if r["amount"] > 20:
        print(r["id"])
```

File handling

python

```
import csv, json

with open("data.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)

with open("data.json") as f:
    data = json.load(f)
```

Error handling

python

```
try:
    value = float(raw)
except ValueError:
    value = 0.0 # fail safe, don't crash the whole pipeline
```

Virtual env + pip

bash

```
python -m venv venv
source venv/bin/activate
pip install pandas numpy requests
pip freeze > requirements.txt
```

pandas essentials

```
python

import pandas as pd
df = pd.read_csv("orders.csv")
df["amount"] = df["amount"].fillna(0)
df = df.drop_duplicates(subset="order_id")
summary = df.groupby("status")["amount"].sum()
df.to_parquet("orders.parquet")
```

Key idea: Python is the glue language of Data Engineering — cleaning, scripting, and connecting every other tool.

2. SQL

Basic query anatomy

```
sql

SELECT status, SUM(amount) AS total
FROM orders
WHERE amount > 0
GROUP BY status
HAVING SUM(amount) > 100
ORDER BY total DESC
LIMIT 10;
```

WHERE filters rows before grouping. **HAVING** filters groups after aggregation.

Joins

```
sql
```

```

SELECT o.order_id, c.email
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;    -- only matches

SELECT c.customer_id, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;        -- all customers, ma

```

CTE + window function

```

sql

WITH totals AS (
    SELECT customer_id, SUM(amount) AS total_spent
    FROM orders GROUP BY customer_id
)
SELECT customer_id, total_spent,
       RANK() OVER (ORDER BY total_spent DESC) AS spend_rank
FROM totals;

```

Quick reference

Clause/Concept	Purpose
DDL ((CREATE), (ALTER))	Define/change structure
DML ((SELECT), (INSERT), (UPDATE), (DELETE))	Read/change data
Primary key	Uniquely identifies a row
Foreign key	Links to another table's primary key
Index	Speeds up lookups on a column
Transaction	Groups statements so they all succeed or all fail ((BEGIN) (COMMIT) (ROLLBACK))

Key idea: SQL is the single most-used skill in Data Engineering — almost every pipeline stage touches it.

3. Database Fundamentals

Relational vs. NoSQL

	Relational (MySQL, Postgres)	NoSQL (MongoDB, Redis, Cassandra)
Schema	Strict, enforced	Flexible
Best for	Structured, related data	Flexible/huge-scale/fast key lookups

ACID (transaction reliability)

- Atomicity — all or nothing
- Consistency — always valid state
- Isolation — concurrent transactions don't interfere
- Durability — committed data survives a crash

CAP theorem (distributed systems): pick 2 of 3 — **C**onsistency, **A**vailability, **P**artition tolerance. Network partitions happen, so you really choose between C and A during one.

Relationships: one-to-one, one-to-many (customer → orders), many-to-many (needs a junction table).

Key idea: Choose relational by default; choose NoSQL for a specific reason (scale, flexible schema, fast key-value lookups).

4. Git

```
bash

git init
git add file.py
git commit -m "Add cleaning logic"

git checkout -b feature/fix-nulls    # new branch
git push origin feature/fix-nulls    # push, then open a Pull Request

git checkout main
git pull origin main
git merge feature/fix-nulls          # combine branches (preserves history)
git rebase main                      # replay commits on top of latest main (lin
```

Conflict markers

```
<<<<<< HEAD
your version
=====
their version
>>>>>> branch-name
```

Fix manually, remove markers, then `git add` + `git commit`.

Rules of thumb:

- Never commit directly to `main` — use a branch + PR.
- Never rebase a branch others are working on.
- Small, focused commits with clear messages.
- `.gitignore` secrets, `venv/`, and large data files.

5. ETL / ELT

```
ETL: Extract → Transform (external engine) → Load
ELT: Extract → Load (raw, into warehouse) → Transform (inside warehouse
with SQL/dbt)
```

	ETL	ELT
Transform happens	Before loading	After loading, in the warehouse
Raw data kept?	Often not	Yes — reprocessable
Typical tools	Legacy engines	dbt + BigQuery/Snowflake

Simple pipeline pattern

```
python

def extract(path): return pd.read_csv(path)
def transform(df):
    df["amount"] = df["amount"].fillna(0)
    return df.drop_duplicates(subset="order_id")
def load(df, path): df.to_csv(path, index=False)
```

Orchestration (Airflow DAG idea)

```
python
```

```
extract_task >> transform_task >> load_task
```

A DAG = tasks + dependencies, no circular loops. Airflow/Cloud Composer handles scheduling, retries, alerting.

dbt = write transformations as SQL (`SELECT`s; dbt builds the dependency graph and runs data quality tests (`unique`, `not_null`)).

Key idea: Modern stacks favor ELT — load raw data cheaply, transform with the warehouse's own compute, always keep the raw copy.

6. Data Warehousing

```
Data Lake          → raw, any format, cheap storage (Cloud Storage/S3)
```

```
Data Warehouse     → structured, modeled, fast analytical queries  
(BigQuery/Snowflake)
```

```
Lakehouse           → lake storage + warehouse-like querying on top
```

OLTP vs OLAP

	OLTP	OLAP
Purpose	Run the business (transactions)	Analyze the business (reporting)
Query shape	Many small reads/writes	Few large scans over history

Star schema (most common)

```
dim_date  
  |  
dim_customer — fact_sales — dim_product
```

Fact table = measurable events (sales amount). Dimension tables = descriptive context (customer, product, date).

Snowflake schema = dimensions normalized further into sub-tables (more joins, less redundancy).

- **Partition** by date → query only scans relevant date segments.
- **Cluster** by a high-cardinality column (e.g., customer_id) → sorts data within partitions for faster filtering.

```
sql

CREATE TABLE fact_orders
PARTITION BY DATE(order_date)
CLUSTER BY customer_id
AS SELECT * FROM staging.orders;
```

Key idea: Star schema + partitioning/clustering = fast, cheap analytical queries at scale.

7. Data Formats

Format	Type	Human-readable	Best for
CSV	Row-based, text	Yes	Small exports, quick sharing
JSON	Row-based, text	Yes	APIs, nested/event data
Parquet	Columnar, binary	No	Data lakes, analytics (default choice)

```
Row-based:    Row1[id,name,amount] Row2[id,name,amount] ...
Columnar:    Column "id":[1,2,3]  Column "amount":[x,y,z] ...
```

Columnar formats let a query like `SUM(amount)` read **only** the amount column — much less I/O than scanning full rows.

```
python

df = pd.read_csv("orders.csv")
df.to_parquet("orders.parquet", compression="snappy") # smaller, faster to qu
```

Key idea: Use CSV/JSON for small/human-facing data; use Parquet for anything analytical or stored in a data lake.

One-Page Summary

Topic	Core Job
Python	Clean, script, glue everything together
SQL	Query and transform data, everywhere
Databases	Store data reliably (relational) or flexibly (NoSQL)
Git	Track, review, and safely ship pipeline code
ETL/ELT	Move data from source → destination, cleanly
Data Warehousing	Model data for fast, trustworthy analytics
Data Formats	Choose the right shape/storage for the job (usually Parquet)